Sesión 2. Análisis exploratorio morfosintáctico

Módulo de Análisis Exploratorio de Datos, Doctorado en Lingüística.

En la sesión anterior repasamos las estructuras de R sobre datos inventados. Ahora trabajamos con un corpus real y anotado, y la pregunta deja de ser cómo funciona una función para pasar a ser qué nos dicen estos datos sobre la lengua.

El apunte tiene dos partes. La primera es un recorrido guiado sobre un treebank del español que haremos en clase. La segunda es una extensión que cada uno aplicará a un texto de su propio interés, anotándolo automáticamente y repitiendo el análisis. El objetivo de esa segunda parte no es solo practicar: es que el procedimiento quede disponible para sus propias investigaciones.

Una advertencia sobre el formato de este documento. A diferencia de la prelectura, aquí casi no se muestran resultados. El código está para ejecutarlo, y los números deben salir de la consola de cada uno. Cuando una pregunta aparece en el texto, es una pregunta genuina cuya respuesta está en los datos.

1. El corpus: UD Spanish AnCora

Trabajaremos con el treebank AnCora del español dentro del proyecto Universal Dependencies. Es un corpus de texto periodístico y de ficción anotado morfosintácticamente, cuya anotación fue revisada manualmente. La versión completa contiene alrededor de 17.700 oraciones y 547.000 ocurrencias (*tokens*).

Usaremos el archivo de desarrollo (*dev*), de unas 1.650 oraciones y 53.000 ocurrencias. Es suficiente para todo lo que haremos y se carga en segundos. Quien quiera trabajar después con el corpus completo solo tiene que cambiar el nombre del archivo.

1.1 Descarga

El corpus está en el repositorio público del proyecto:

https://github.com/UniversalDependencies/UD_Spanish-AnCora

Descarga el archivo `es_ancora-ud-dev.conllu` y guárdalo en la carpeta de tu proyecto de RStudio. Puedes hacerlo desde el navegador o desde R:

```
url_base <- "https://raw.githubusercontent.com/UniversalDependencies/UD_Spanish-AnCora/maste
download.file(paste0(url_base, "es_ancora-ud-dev.conllu"),
              destfile = "es_ancora-ud-dev.conllu")
```

Si la descarga automática falla por restricciones de red, usa el navegador. El archivo pesa unos pocos megabytes.

1.2 Paquetes necesarios

```
install.packages("udpipe")   # solo la primera vez
library(udpipe)
```

El paquete `udpipe` cumple dos funciones en esta sesión: leer archivos en formato CoNLL-U y, más adelante, anotar texto propio.

1.3 El formato CoNLL-U

Antes de cargar nada, conviene entender qué se está cargando. CoNLL-U es el formato estándar de anotación morfosintáctica. Cada línea corresponde a una ocurrencia y contiene diez campos separados por tabuladores; las oraciones se separan por líneas en blanco y los comentarios empiezan con almohadilla.

Los campos que usaremos son estos:

Campo	Contenido
`token_id`	Posición de la ocurrencia dentro de la oración
`token`	La forma tal como aparece en el texto
`lemma`	La forma de diccionario
`upos`	Categoría gramatical universal (NOUN, VERB, ADJ, . . .)
`xpos`	Etiqueta específica de la lengua
`feats`	Rasgos morfológicos, separados por barras verticales
`head_token_id`	Posición del núcleo del que depende esta ocurrencia
`dep_rel`	Relación de dependencia con ese núcleo

Un detalle que importará más adelante: cuando `head_token_id` vale 0, la ocurrencia es la raíz de la oración y no depende de ninguna otra.

Puedes mirar el archivo crudo antes de procesarlo. Vale la pena hacerlo una vez:

```
writeLines(head(readLines("es_ancora-ud-dev.conllu", n = 30)))
```

1.4 Carga

```
ancora <- udpipe_read_conllu("es_ancora-ud-dev.conllu")
```

La función devuelve un marco de datos (*data frame*) ordinario. Todo lo de la prelectura se aplica sin adaptación.

2. Primera inspección

Antes de preguntarle nada a los datos, hay que saber qué hay.

```
str(ancora)
dim(ancora)
```

```r
names(ancora)
head(ancora[, c("sentence_id", "token_id", "token", "lemma", "upos", "feats")], 15)
```

Cuatro preguntas de inventario que conviene responder siempre al abrir un corpus nuevo:

```r
# Cuántas ocurrencias
nrow(ancora)

# Cuántas oraciones
length(unique(ancora$sentence_id))

# Cuántas categorías gramaticales distintas
length(unique(ancora$upos))

# Dónde hay valores faltantes
colSums(is.na(ancora))
```

La última merece atención. El campo `feats` estará vacío en las categorías que no tienen morfología flexiva, y eso no es un error del corpus sino una propiedad del sistema. Identificar qué categorías son conviene hacerlo ahora y no cuando un promedio salga raro.

```r
table(ancora$upos, is.na(ancora$feats))
```

Actividades 2

2.1 Carga el corpus y responde: número de ocurrencias, número de oraciones, número de tipos léxicos distintos en `token` y en `lemma`. ¿Por qué los dos últimos números difieren, y en qué dirección?

2.2 Usando `table()` sobre `upos`, identifica cuántas categorías gramaticales distintas hay. Compáralo con el inventario oficial de Universal Dependencies, que tiene diecisiete. ¿Falta alguna en este corpus? Si es así, propón una explicación.

2.3 Calcula la razón tipo-ocurrencia (*type-token ratio*) del corpus sobre `token` y sobre `lemma`. Reflexiona sobre cuál de las dos medidas responde mejor a la pregunta por la riqueza léxica.

3. Distribución de categorías gramaticales

Esta es la variable morfosintáctica más básica y la puerta de entrada al análisis. Es nominal, de modo que las herramientas son tablas de frecuencia y gráficos de barras.

```r
frec_upos <- table(ancora$upos)
frec_upos

# En proporciones
```

```r
prop_upos <- round(prop.table(frec_upos) * 100, 2)
sort(prop_upos, decreasing = TRUE)
```

Para visualizarlo conviene ordenar las barras por frecuencia, porque el orden alfabético no dice nada:

```r
barplot(sort(frec_upos, decreasing = TRUE),
        las = 2,
        cex.names = 0.8,
        main = "Distribución de categorías gramaticales",
        ylab = "Ocurrencias")
```

Dos observaciones para discutir en clase. La primera: la puntuación (`PUNCT`) suele aparecer entre las categorías más frecuentes, lo que obliga a decidir si forma parte del análisis o no. No hay una respuesta correcta, pero sí hay que tomar la decisión de forma explícita y documentarla. La segunda: la distribución está fuertemente sesgada, igual que la de las palabras, aunque el inventario de categorías sea pequeño y cerrado.

3.1 Densidad léxica

Una medida clásica que se deriva directamente de esta distribución es la densidad léxica: la proporción de palabras de contenido sobre el total.

```r
contenido <- c("NOUN", "VERB", "ADJ", "ADV", "PROPN")

ancora$es_contenido <- ancora$upos %in% contenido

mean(ancora$es_contenido)      # densidad léxica global
```

Recuerda por qué esto funciona: `%in%` devuelve un vector lógico, y la media de un vector lógico es la proporción de valores verdaderos. Es el mismo mecanismo que la suma que vimos en la prelectura.

La densidad léxica cobra interés cuando se compara entre textos o registros, no en abstracto. Volveremos a ella en la sección 5.

Actividades 3

3.1 Genera la tabla de proporciones de `upos` y ordénala de mayor a menor. ¿Qué tres categorías concentran la mayor parte de las ocurrencias?

3.2 Repite el gráfico de barras excluyendo la puntuación. Compara ambas versiones. ¿Cambia la interpretación de la distribución?

3.3 Calcula la densidad léxica global. Luego calcúlala excluyendo `PUNCT` del denominador. Explica cuál de las dos cifras es más comparable con la de otro corpus y por qué.

3.4 Los determinantes (`DET`) y las adposiciones (`ADP`) son categorías cerradas y muy frecuentes. Calcula qué proporción del corpus ocupan entre ambas y

comenta el resultado a la luz de la ley de Zipf.

4. Rasgos morfológicos

El campo `feats` contiene la información flexiva codificada como pares atributo-valor separados por barras verticales. Una forma verbal puede aparecer, por ejemplo, como `Mood=Ind|Number=Sing|Person=3|Tense=Pres|VerbForm=Fin`.

Extraer esa información es el primer problema real de manipulación de datos de la sesión, y se resuelve con las funciones de búsqueda de patrones que vimos en la prelectura.

4.1 Extraer un rasgo

Para saber si una ocurrencia tiene un valor determinado basta `grepl()`:

```
# Ocurrencias marcadas como femeninas
ancora$femenino <- grepl("Gender=Fem", ancora$feats)

table(ancora$femenino)
```

Para extraer el valor del rasgo, no solo su presencia, se usa una expresión regular con grupo de captura:

```
extraer_rasgo <- function(feats, rasgo) {
  patron <- paste0(rasgo, "=([A-Za-z]+)")
  valor <- regmatches(feats, regexpr(patron, feats))
  valor <- sub(paste0(rasgo, "="), "", valor)
  # Devolver NA donde no hubo coincidencia
  resultado <- rep(NA_character_, length(feats))
  resultado[grepl(patron, feats)] <- valor
  resultado
}

ancora$genero <- extraer_rasgo(ancora$feats, "Gender")
ancora$numero <- extraer_rasgo(ancora$feats, "Number")
ancora$modo   <- extraer_rasgo(ancora$feats, "Mood")
ancora$tiempo <- extraer_rasgo(ancora$feats, "Tense")

table(ancora$genero, useNA = "ifany")
```

El argumento `useNA = "ifany"` hace visibles los valores faltantes en la tabla. Sin él, `table()` los oculta, lo que puede llevar a interpretar mal una distribución.

El paquete `udpipe` ofrece un atajo que hace algo equivalente para todos los rasgos a la vez:

```
ancora_morf <- cbind_morphological(ancora, term = "feats")
names(ancora_morf)
```

Vale la pena haber escrito la función manual antes de usar el atajo, porque el problema de extraer información estructurada desde un campo de texto reaparecerá con otros corpus que no tienen paquete dedicado.

4.2 Distribuciones morfológicas

Con los rasgos extraídos, las preguntas se vuelven específicamente lingüísticas.

```
# Género y número, solo en sustantivos
sust <- subset(ancora, upos == "NOUN")

table(sust$genero, useNA = "ifany")
table(sust$numero, useNA = "ifany")

# Cruce de ambos
table(sust$genero, sust$numero)
```

En los verbos, la distribución de modo y tiempo:

```
verbos <- subset(ancora, upos == "VERB")

sort(table(verbos$modo), decreasing = TRUE)
sort(table(verbos$tiempo), decreasing = TRUE)
```

La proporción de subjuntivo respecto de indicativo es una cifra que suele sorprender. Antes de calcularla, anota tu estimación intuitiva; después compárala con el dato. Es un buen ejemplo de por qué la intuición lingüística necesita verificación empírica, que era el argumento de la primera sesión.

Actividades 4

4.1 Extrae el rasgo `Person` y tabula su distribución en las formas verbales finitas (`VerbForm=Fin`). ¿Qué persona domina, y qué relación tiene ese resultado con el tipo de texto del corpus?

4.2 Construye la tabla de contingencia entre género y número en los adjetivos (`ADJ`). Convierte la tabla a proporciones por filas con `prop.table(tabla, margin = 1)` e interpreta el resultado en una frase.

4.3 Calcula la proporción de formas verbales en subjuntivo sobre el total de formas con modo marcado. Contrasta la cifra con tu estimación previa.

4.4 Compara la distribución de número (singular frente a plural) entre sustantivos y adjetivos. ¿Son similares? Propón una explicación para la diferencia, si la hay.

4.5 El rasgo `VerbForm` distingue formas finitas, infinitivos, gerundios y participios. Tabula su distribución y calcula qué proporción de las formas verbales del corpus son no finitas.

5. Del token a la oración

Hasta aquí cada fila era una ocurrencia. Muchas preguntas morfosintácticas, sin embargo, se formulan a nivel de oración: cuán larga es, cuántos verbos contiene, qué densidad léxica tiene. Eso requiere agregar (*aggregate*).

5.1 Longitud de oración

```
long_oracion <- aggregate(token_id ~ sentence_id,
                          data = ancora,
                          FUN = length)
names(long_oracion)[2] <- "n_tokens"

summary(long_oracion$n_tokens)

hist(long_oracion$n_tokens,
     breaks = 40,
     main = "Distribución de la longitud de oración",
     xlab = "Ocurrencias por oración",
     ylab = "Frecuencia")
```

Observa la forma del histograma. La longitud de oración es una variable de conteo, y su distribución suele estar sesgada hacia la derecha: muchas oraciones cortas o medianas y una cola larga de oraciones muy largas.

Esto tiene una consecuencia directa sobre qué estadístico de resumen usar:

```
mean(long_oracion$n_tokens)
median(long_oracion$n_tokens)
```

Si ambos valores difieren de forma apreciable, la media está siendo arrastrada por la cola y la mediana describe mejor la oración típica. Es exactamente el argumento que veremos formalizado en la sesión sobre medidas de tendencia central.

El diagrama de caja hace visibles los valores atípicos:

```
boxplot(long_oracion$n_tokens,
        horizontal = TRUE,
        main = "Longitud de oración",
        xlab = "Ocurrencias")
```

Inspeccionar los casos extremos es siempre informativo. Puede tratarse de oraciones genuinamente largas o de errores de segmentación:

```
oracion_larga <- long_oracion$sentence_id[which.max(long_oracion$n_tokens)]
paste(subset(ancora, sentence_id == oracion_larga)$token, collapse = " ")
```

5.2 Composición gramatical por oración

Podemos calcular, para cada oración, cuántas ocurrencias de cada categoría contiene:

```
comp <- as.data.frame.matrix(table(ancora$sentence_id, ancora$upos))
comp$sentence_id <- rownames(comp)
comp$n_tokens <- rowSums(comp[, setdiff(names(comp), "sentence_id")])

head(comp[, c("sentence_id", "NOUN", "VERB", "ADJ", "n_tokens")])
```

Con esto se derivan medidas de interés. La densidad verbal, por ejemplo, es una aproximación al número de cláusulas y por tanto a la complejidad oracional:

```
comp$densidad_verbal <- comp$VERB / comp$n_tokens
comp$ratio_nom_verb  <- comp$NOUN / comp$VERB

summary(comp$densidad_verbal)
```

La razón entre sustantivos y verbos es una medida clásica para distinguir estilos: los textos de estilo nominal, característico de la prosa académica y administrativa, presentan valores más altos que los de estilo verbal, propio de la narración.

Cuidado con un detalle: si una oración no contiene verbos, la razón será infinita o indefinida. Verifícalo antes de calcular promedios.

```
sum(comp$VERB == 0)
```

Actividades 5

5.1 Calcula la longitud media y mediana de oración. Interpreta la diferencia entre ambas cifras.

5.2 Construye el histograma de la longitud de oración y luego el mismo histograma restringido a oraciones de menos de cincuenta ocurrencias. ¿Qué se ve mejor en cada uno?

5.3 Identifica las cinco oraciones más largas del corpus y reconstruye su texto. ¿Son oraciones legítimas o hay problemas de segmentación?

5.4 Calcula la densidad léxica por oración, no global. Obtén su distribución y compárala con la cifra global de la actividad 3.3. ¿Por qué pueden diferir?

5.5 Calcula la razón entre sustantivos y verbos por oración, excluyendo las oraciones sin verbos. Obtén su mediana y examina las cinco oraciones con la razón más alta. ¿Qué tienen en común?

6. Relaciones de dependencia

La anotación no solo asigna categorías: también establece qué depende de qué. Esto abre un tipo de análisis que no está disponible en un corpus etiquetado únicamente por categoría gramatical.

6.1 Distribución de relaciones

```
frec_dep <- sort(table(ancora$dep_rel), decreasing = TRUE)
head(frec_dep, 15)

round(prop.table(frec_dep)[1:10] * 100, 2)
```

6.2 Distancia de dependencia

La distancia de dependencia es el número de posiciones que separan a una palabra de su núcleo sintáctico. Es una medida con respaldo empírico considerable en psicolingüística: distancias mayores implican mayor carga de memoria de trabajo, y las lenguas parecen tender a minimizarlas.

```
# Los identificadores vienen como texto; hay que convertirlos
ancora$tok_num  <- as.numeric(ancora$token_id)
ancora$head_num <- as.numeric(ancora$head_token_id)

# La raíz de cada oración tiene head 0 y debe excluirse
ancora$dist_dep <- ifelse(ancora$head_num == 0,
                          NA,
                          abs(ancora$head_num - ancora$tok_num))

summary(ancora$dist_dep)

hist(ancora$dist_dep[!is.na(ancora$dist_dep)],
     breaks = 50,
     main = "Distribución de la distancia de dependencia",
     xlab = "Distancia en ocurrencias")
```

La distribución resultante es fuertemente asimétrica: la mayoría de las dependencias son muy cortas. Ese es precisamente el hallazgo que sostiene la hipótesis de minimización de distancia.

También se puede examinar la dirección de la dependencia, es decir, si el núcleo precede o sigue al dependiente:

```
ancora$direccion <- ifelse(is.na(ancora$dist_dep), NA,
                    ifelse(ancora$head_num > ancora$tok_num,
                           "núcleo a la derecha",
                           "núcleo a la izquierda"))

table(ancora$direccion)
```

6.3 Cruce entre variables

Ahora podemos preguntar si la distancia depende del tipo de relación:

```r
dist_por_rel <- aggregate(dist_dep ~ dep_rel,
                          data = ancora,
                          FUN = median)

dist_por_rel[order(-dist_por_rel$dist_dep), ][1:10, ]
```

Y visualizarlo comparando distribuciones completas, no solo medianas:

```r
relaciones <- c("det", "amod", "nsubj", "obj", "obl", "advmod")
sub_rel <- subset(ancora, dep_rel %in% relaciones)

boxplot(dist_dep ~ dep_rel,
        data = sub_rel,
        main = "Distancia de dependencia por tipo de relación",
        xlab = "", ylab = "Distancia",
        las = 2)
```

Este gráfico es el primero de la sesión que responde a una pregunta genuinamente bivariada: cómo se relaciona una variable numérica con una categórica.

Actividades 6

6.1 Obtén las diez relaciones de dependencia más frecuentes con sus proporciones. ¿Cuáles esperabas encontrar arriba y cuáles te sorprenden?

6.2 Calcula la distancia de dependencia media y mediana del corpus. Compara ambas e interpreta la diferencia a la luz de la forma de la distribución.

6.3 Compara la proporción de dependencias con núcleo a la izquierda frente a núcleo a la derecha. Relaciona el resultado con lo que sabes sobre el orden de constituyentes del español.

6.4 Genera el diagrama de cajas de la distancia por tipo de relación. ¿Qué relaciones muestran las distancias más largas? Propón una explicación sintáctica.

6.5 Examina si la distancia de dependencia se relaciona con la longitud de la oración. Agrega la distancia media por oración, únela mediante `merge()` con la tabla de longitudes y construye un diagrama de dispersión. Describe el patrón sin afirmar causalidad.

7. Extensión: tu propio texto

Todo lo anterior se aplica a cualquier texto en español, porque `udpipe` incluye modelos entrenados que producen exactamente la misma estructura de datos.

Esta parte es individual y se entrega junto con las actividades. Elige un texto relacionado con tu investigación: un fragmento de tu corpus de trabajo, un texto de un género que te interese, o dos textos que quieras comparar.

7.1 Descargar y cargar el modelo

```
library(udpipe)

# Solo la primera vez: descarga el modelo al directorio de trabajo
modelo_archivo <- udpipe_download_model(language = "spanish-ancora")

# En sesiones posteriores basta cargar el archivo ya descargado
modelo <- udpipe_load_model(modelo_archivo$file_model)
```

El modelo pesa alrededor de quince megabytes y queda guardado en tu carpeta. No hace falta volver a descargarlo.

7.2 Anotar

```
mi_texto <- readLines("mi_texto.txt", encoding = "UTF-8")

anotado <- udpipe_annotate(modelo, x = mi_texto)
anotado <- as.data.frame(anotado)

str(anotado)
```

Compara la salida con la estructura de `ancora`. Las columnas son las mismas, de modo que todo el código de las secciones 3 a 6 funciona sin cambios sobre este nuevo objeto.

Si tienes varios textos y quieres conservar su identidad, pásalos como un vector con nombres o usa el argumento `doc_id`:

```
textos <- c(t1 = "Primer texto completo...",
            t2 = "Segundo texto completo...")

anotado <- as.data.frame(udpipe_annotate(modelo,
                                         x = textos,
                                         doc_id = names(textos)))
```

La columna `doc_id` permitirá después comparar textos con `aggregate()`.

7.3 Una advertencia necesaria

La anotación automática tiene errores. La precisión de estos modelos en categoría gramatical suele ser alta, pero no perfecta, y baja en dependencias sintácticas. En AnCora la anotación fue revisada a mano; en tu texto no.

Esto no invalida el análisis, pero cambia lo que se puede afirmar. Dos consecuencias prácticas. La primera: inspecciona manualmente una muestra de la anotación antes de confiar en las cifras agregadas.

```
set.seed(42)
muestra <- anotado[sample(nrow(anotado), 30), c("token", "lemma", "upos", "dep_rel")]
```

`muestra`

La segunda: si tu texto pertenece a un registro muy distinto del que entrenó el modelo, que fue prensa y ficción escritas, la tasa de error será mayor. Transcripciones de habla espontánea, lenguaje de redes sociales o textos históricos son casos donde conviene ser especialmente cauteloso.

7.4 Qué entregar

Un guion de código (*script*) y un informe breve, de no más de dos páginas, con esta estructura:

Descripción del texto. Qué es, de dónde viene, por qué lo elegiste, cuántas ocurrencias y oraciones tiene.

Tres preguntas exploratorias. Formuladas antes de mirar los resultados, no después.

Análisis. Al menos un análisis univariado de una variable categórica, uno de una variable numérica y un cruce entre dos variables. Con sus gráficos correspondientes.

Hallazgos. Qué encontraste, incluyendo lo que no esperabas. Si algo contradijo tu intuición inicial, dilo explícitamente: ese es el resultado más valioso de un análisis exploratorio.

Limitaciones. Qué no puedes concluir a partir de estos datos, y qué habría que hacer para poder concluirlo. Menciona aquí la calidad de la anotación automática y el problema de la representatividad del texto elegido.

8. Cierre

Lo que hicimos en esta sesión responde a una estructura que se repetirá en cualquier análisis exploratorio, con independencia del corpus.

Primero, inventario: qué hay, cuánto hay, qué falta. Segundo, distribuciones univariadas, empezando por las categóricas y siguiendo por las numéricas, atendiendo siempre a la forma de la distribución antes de resumirla con un solo número. Tercero, agregación a otro nivel de análisis, en este caso de la ocurrencia a la oración. Cuarto, relaciones entre variables.

Y a lo largo de todo el proceso, decisiones que hay que documentar: si la puntuación entra o no en el análisis, qué hacer con las oraciones sin verbos, qué categorías cuentan como palabras de contenido. Ninguna de esas decisiones es obvia y todas afectan los resultados. La diferencia entre un análisis exploratorio y una serie de gráficos sueltos está, en buena medida, en si esas decisiones quedaron registradas.

Referencias

Nivre, J. et al. (2020). Universal Dependencies v2: An Evergrowing Multilingual Treebank Collection. *Proceedings of LREC 2020*. Disponible en universaldependencies.org.

Taulé, M., Martí, M. A. y Recasens, M. (2008). AnCora: Multilevel Annotated Corpora for Catalan and Spanish. *Proceedings of LREC 2008*.

Wijffels, J. (2023). *udpipe: Tokenization, Parts of Speech Tagging, Lemmatization and Dependency Parsing with the UDPipe NLP Toolkit*. Paquete de R. Documentación en bnosac.github.io/udpipe.

Liu, H. (2008). Dependency distance as a metric of language comprehension difficulty. *Journal of Cognitive Science*, 9(2), 159-191.

Gries, S. Th. (2009). *Quantitative Corpus Linguistics with R*. Routledge.